

Assignment 3

Spencer Collins

06/02/2026

Intro To C Programming

3.1: Read the documentation on strcpy and strncpy. What is the main difference between these functions? Why might you use one and not the other?

Strcpy copies the entire string from the source to the destination, including the null terminator. Strncpy only copies the specified number of bytes in a string from the source to the destination. You would use strcpy when you want to copy the whole string and strncpy when you would want to copy only a substring.

3.2: Is this legal code? Why or why not? What does it do?

```
void mystery(char* x) {  
    while (*x) {  
        if (*x > 96) *x -= 32;  
        x++;  
    }  
}
```

This code is a lowercase to uppercase ASCII converter. While the code is iterating through the character array, it is checking if the character is lowercase. If it is, then it subtracts 32 from it to get the uppercase version of the character. Then it iterates the pointer to the next character in the array.

3.3: Compile and run this program:

```
#include "stdio.h"  
  
void print_size(int a[]) {  
    printf("sizeof(a) = %lu\n", sizeof(a));  
}  
  
int main(int argc, char** argv) {  
    int a[7];  
    printf("sizeof(a) = %lu\n", sizeof(a));  
    print_size(a);  
    return 0;  
}
```

What does it do? Why does it give different sizeof values in different places?

This program defines a function to print the size of an integer array and then defines an integer array that holds 7 integers and then prints the size of the initialized array using

the `sizeof()` function and calling the `print_size()` function. The printed `sizeof()` for `int a[7]` is different from `print_size(a)` since the argument for `print_size()`, `int a[]`, decays to a pointer, so it prints the byte size of the pointer, and not the byte size of the whole array in memory.

Week 3 Project

Part A:

Build a command-line tool that sums its arguments, which will all be integers, and prints the result to the console. **It should work on all platforms.** An example session might look like this:

```
$ ./sum
0
$ ./sum 1 2 3
6
$ ./sum 61 18 -42 100
137
```

All arguments will be between -1000 and 1000, inclusive; there can be as many as 100 of them.

Part B:

Extend your tool to handle Roman numerals, which may be mixed in with regular numeric arguments; the minus sign prefix may be applied to them, and letters may be lower or upper case.

```
$ ./sum XIV IX L
73
$ ./sum -x 43 iii
36
```

Standard library functions that may be helpful include `atoi` in `<stdlib.h>` and `strlen` in `<string.h>`.

3.4: Include your code for this project in your PDF.

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>
```

```
int main(int argc, char* argv[]) {  
    int sum = 0;  
    if (argc < 101) {  
        for (int i =1; i<argc; i++) {  
            char* numbers = "1234567890";  
            int num = 0;  
            char* arg = argv[i];  
  
            int length = strlen(arg);  
  
            for (int j=0;j<length;j++) {  
                if (arg[j] == 'i' || arg[j] == 'I') {  
                    num+=1;  
                } else if (arg[j] == 'v' || arg[j] == 'V') {  
                    num+=5;  
                } else if (arg[j] == 'x' || arg[j] == 'X') {  
                    num+=10;  
                } else if(arg[j] == 'l' || arg[j] == 'L') {  
                    num += 50;  
                } else {  
                    for (int k=0;k<11;k++) {  
                        if (arg[j] == numbers[k]) {  
                            num = atoi(arg);  
                            break;  
                        }  
                    }  
                }  
            }  
        }  
    }  
}
```

```

        }

    }

}

    if ((arg[length-2] == 'i' || arg[length-2] == 'I') && ((arg[length-1]
== 'v' || arg[length-1] == 'V') || (arg[length-1] == 'x' || arg[length-1] ==
'X')))) {

        num-=2;

    }


    if (arg[0] == '-') {

        num *= -1;

    }


    if (num >= -1000 && num <= 1000) {

        sum += num;

    }

}

printf("%d", sum);
} else {

    printf("%s\n", "Number of arguments > 100!");

}


return 0;
}

```